MODULE 14

Networking & Persistence

Retrofit, Coroutines, Room, Secure Storage, and Offline Sync

Phase 03 | Android Development with Kotlin

What You Will Build in This Module

1

Networking

Retrofit, kotlinx.serialization, coroutines, interceptors, MockWebServer

2

Persistence and Security

Room, migrations, DataStore, Keystore encryption, token refresh, pinning

3

Offline-First

Single source of truth, outbox, WorkManager sync, capstone



©Logical Imagination Group LLC

Section 1

Networking: Retrofit, JSON, and Coroutines

1.1 HTTP, REST, and Retrofit Foundations

1.2 JSON Serialization with kotlinx.serialization

1.3 Coroutines for Asynchronous Networking

1.4 Interceptors, Error Handling, and Testing the Network Layer

HTTP, REST, and Retrofit Foundations

- Read an HTTP request and response and name its verb, path, headers, status code, and body
- Configure Retrofit and OkHttp in a Gradle project with the correct permission and base URL
- Declare a typed API interface using Retrofit annotations and suspend functions



©Logical Imagination Group LLC

Anatomy of a Mobile API Call

- Verb says intent:
  - GET reads
  - POST creates
  - PUT replaces
  - DELETE removes
- Status code says outcome:
  - 2xx success
  - 4xx client fault
  - 5xx server fault
- Headers carry identity and format: Authorization, Content-Type, Accept
- Always HTTPS; the transport protects the user, not your code

Code Bite

```
GET /v1/accounts HTTP/1.1
Host: api.pncmobile.example
Authorization: Bearer eyJhbGciOi...
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: private, max-age=30

[{"id":"acc_1","name":"Virtual
Wallet","masked_number":"****4821","balance"
:2450.75}]
```



©Logical Imagination Group LLC

Retrofit and OkHttp: Division of Labor

OkHttp moves bytes: connections, TLS, timeouts, interceptors, cache

Retrofit turns an annotated interface into OkHttp calls

A converter turns JSON bodies into Kotlin objects and back

One shared client per app, never one per request



©Logical Imagination Group LLC

Gradle Setup

- Retrofit 3.0, OkHttp 5.5, kotlinx.serialization converter (already included earlier)
- Serialization plugin applied in the module build file
- INTERNET permission is declared, never requested at runtime
- Version catalog keeps versions in one place

Code Bite

```
plugins {
  alias(libs.plugins.kotlin.serialization)
}
dependencies {
  implementation(libs.retrofit
  implementation(libs.okhttp)
  implementation(libs.okhttp.logging)
}
```



©Logical Imagination Group LLC

Gradle Permissions

- The INTERNET permission MUST be included in AndroidManifest.xml
- If not, the requests will just silently fail
  - Which may cause errors downstream in your app, of course!

Code Bite

```
// AndroidManifest.xml
<uses-permission android:name =
            "android.permission.INTERNET" />
```



©Logical Imagination Group LLC

Declaring the API Interface

- One annotation per HTTP verb: @GET, @POST, @PUT, @DELETE
- @Path fills the URL, @Query appends parameters, @Body sends JSON
- Mark functions suspend so callers use coroutines
- Return the body type, or Response<T> when you need the status code

Code Bite

```
interface AccountsApi {
  @GET("accounts")
    suspend fun getAccounts():
List<AccountDto>


  @GET("accounts/{id}/transactions")
  suspend fun getTransactions(
    @Path("id") accountId: String,
    @Query("limit") limit: Int = 50,
    @Query("cursor") cursor: String? = null
  ): TransactionPageDto
}
```



©Logical Imagination Group LLC

Sending Data

- @Path fills the URL, @Query appends parameters, @Body sends JSON
- Mark functions suspend so callers use coroutines
- Return the body type, or Response<T> when you need the status code

Code Bite

```
interface AccountsApi {
  . . .

  @POST("transfer")
    suspend fun transfer(@Body request:
        TransferRequestDto): Response<Unit>

}
```



©Logical Imagination Group LLC

Building the Client and Reaching Localhost

- Base URL must end with a slash
- Emulator reaches your computer at 10.0.2.2, not localhost
- Allow cleartext only in a debug network security config
- Release builds stay HTTPS only

Code Bite

```
val json = Json { ignoreUnknownKeys = true }

val retrofit = Retrofit.Builder()
  .baseUrl(BuildConfig.API_BASE_URL)
  .client(okHttpClient)
  .addConverterFactory(
     json.asConverterFactory(
       "application/json".toMediaType())
     ).build()

val accountsApi: AccountsApi =
   retrofit.create(AccountsApi::class.java)
```



©Logical Imagination Group LLC

Lesson 1.1

Skill Check

1. A response returns status 401. Which layer of the app should react first?

A) The Room database
B) The authentication layer, which should attempt a token refresh
C) The Compose theme
D) The Gradle build

2. Why does the emulator need 10.0.2.2 instead of localhost?

A) It is faster
B) localhost on the emulator points to the emulator itself
C) Android forbids localhost for security
D) Retrofit rewrites it

3. Which configuration belongs on OkHttpClient rather than Retrofit?

A) @GET paths
B) Read timeout
C) Return types
D) Query parameter names

4. Where should cleartext HTTP be allowed?

A) In the main manifest for all builds
B) Only in a debug-only network security config
C) In every build type
D) Nowhere, ever, including debug



©Logical Imagination Group LLC

JSON Serialization with kotlinx.serialization

- Explain why network DTOs are separate from domain models
- Represent money and timestamps safely
- Annotate DTOs with @Serializable and @SerialName and configure Json for forward compatibility
- Handle unknown enum values without crashing



©Logical Imagination Group LLC

DTO, Entity, Domain: Three Different Shapes

- DTO mirrors the server JSON exactly
- Domain model mirrors what the app needs
- Entity mirrors a database row (later lesson)
- Mappers are the only code that knows two shapes at once



©Logical Imagination Group LLC

DTO, Entity, Domain: Three Different Shapes (code)

```
@Serializable
data class AccountDto(
    val id: String,
    val name: String,
    @SerialName("masked_number") val maskedNumber: String,
    @SerialName("balance_cents") val balanceCents: Long
)
data class Account(
    val id: String,
    val name: String,
    val maskedNumber: String,
    val balance: Money
)
fun AccountDto.toDomain() = Account(id, name, maskedNumber, Money(balanceCents))
```



©Logical Imagination Group LLC

Json Configuration

- Defaults make fields optional in the payload
- ignoreUnknownKeys lets the server add fields safely

Code Bite

```
val appJson = Json {
    ignoreUnknownKeys = true
    explicitNulls = false
    coerceInputValues = true
    encodeDefaults = false
}
```



©Logical Imagination Group LLC

Annotations for Json Encoding

- @Serializable generates the serializer at compile time
- @SerialName maps snake_case JSON to camelCase Kotlin

Code Bite

```
@Serializable
data class TransferRequestDto(
  @SerialName("from_account")
     val fromAccountId: String,
  @SerialName("to_account")
     val toAccountId: String,
  @SerialName("amount_cents")
     val amountCents: Long,
     val memo: String? = null
)
```



©Logical Imagination Group LLC

Money Done Safely

- Never use Double for currency
  - 0.1 + 0.2 is not 0.3
- Safe (accurate) options include:
  - Integer minor units (cents)
  - BigDecimal strings
- It can be useful to wrap either of those options in a class

Code Bite

```
class Money(val cents: Long) {

   operator fun plus(o: Money) =
                     Money(cents + o.cents)

   fun format(
      locale: Locale = Locale.getDefault()
   ): String =
      NumberFormat
            .getCurrencyInstance(locale)
            .format(cents / 100.0)
}
```



©Logical Imagination Group LLC

Money and Time Done Safely

- Parse ISO-8601 timestamps to Instant with a custom serializer
- Format for display only at the UI edge, using the user's locale

Code Bite

```
object InstantSerializer:
                    KSerializer<Instant> {
  override val descriptor =
    PrimitiveSerialDescriptor("Instant",
                        PrimitiveKind.STRING)
    override fun serialize(encoder: Encoder,
                        value: Instant) =
    encoder.encodeString(value.toString())
    override fun deserialize(
      decoder: Decoder
    ): Instant =
    Instant.parse(decoder.decodeString())
}
```



©Logical Imagination Group LLC

Enums, Sealed Types, and Unknown Values

A new server enum value must not crash an old app

coerceInputValues maps unknown enums to the declared default

Add an explicit UNKNOWN case and render it gracefully

Polymorphic payloads use a sealed class with a discriminator



©Logical Imagination Group LLC

Enums, Sealed Types, and Unknown Values (code)

```
@Serializable
enum class TxType {
    @SerialName("debit") DEBIT,
    @SerialName("credit") CREDIT,
    @SerialName("hold") HOLD,
    UNKNOWN
}
@Serializable
data class TransactionDto(
    val id: String,
    @SerialName("amount_cents") val amountCents: Long,
    val type: TxType = TxType.UNKNOWN,
    @Serializable(with = InstantSerializer::class) val posted: Instant
)
```



©Logical Imagination Group LLC

Lesson 1.2

Skill Check ✓

1. Why not let Composables read DTOs directly?

A) DTOs are slower
B) Wire format changes would ripple through the UI
C) Compose cannot display DTOs
D) DTOs cannot be null

2. What does ignoreUnknownKeys = true protect against?

A) Missing required fields
B) New fields added by the server
C) Slow networks
D) Expired tokens

3. Which type is safest for currency amounts?

A) Float
B) Double
C) Long cents or BigDecimal
D) String concatenation

4. How do you survive a new enum value from the server?

A) Catch the exception in the Composable
B) Declare an UNKNOWN default and enable coerceInputValues
C) Delete the enum
D) Ask the server team never to change it



©Logical Imagination Group LLC

Section 2

Persistence and Security: Room, DataStore, Keystore, Auth

2.1 Room Fundamentals: Entities, DAOs, Database

2.2 Relations, Transactions, and Migrations

2.3 DataStore and Secure Storage with the Android Keystore

2.4 Authentication Plumbing: Tokens, Refresh, Pinning, Logout

Room Fundamentals: Entities, DAOs, Database

- Choose Room over raw SQLite and explain its three building blocks
- Write DAOs with suspend queries, reactive Flow queries, and @Upsert
- Define entities with primary keys, indexes, and type converters
- Test a DAO against an in-memory database



©Logical Imagination Group LLC

Why Room, and Its Three Parts

- Entity: a class that maps to a table
- DAO: an interface of queries, verified at compile time
- Database: the holder that wires entities and DAOs
- Room checks your SQL during the build, not in production

Code Bite

```
// build.gradle.kts
plugins { alias(libs.plugins.ksp) }
dependencies {
    implementation(libs.room.runtime)
    implementation(libs.room.ktx)
    ksp(libs.room.compiler)
    testImplementation(libs.room.testing)
}
ksp { arg("room.schemaLocation",
"$projectDir/schemas") }
```



©Logical Imagination Group LLC

Entities

- @Entity with a primary key; use string ids from the server
- @ColumnInfo renames columns; @Index speeds lookups

Code Bite

```
@Entity(tableName = "accounts")
data class AccountEntity(
    @PrimaryKey val id: String,
    val name: String,
    @ColumnInfo(name = "masked_number") val maskedNumber: String,
    @ColumnInfo(name = "balance_cents") val balanceCents: Long,
    @ColumnInfo(name = "updated_at") val updatedAtEpochMs: Long
)
```



©Logical Imagination Group LLC

Type Converters

- SQLite stores primitives only: convert Instant and enums
- Keep entities flat and boring; richer types belong in the domain

Code Bite

```
class Converters {

    @TypeConverter fun fromInstant(v:
Instant?): Long? = v?.toEpochMilli()

    @TypeConverter fun toInstant(v: Long?):
Instant? = v?.let(Instant::ofEpochMilli)

}
```



©Logical Imagination Group LLC

DAOs: Queries, Flow, and Upsert

suspend functions for one-shot reads and writes

Flow return type makes a query observable

@Upsert inserts or updates in one call

Room runs suspend and Flow queries off the main thread



©Logical Imagination Group LLC

DAOs: Queries, Flow, and Upsert (code)

```
@Dao
interface AccountDao {
    @Query("SELECT * FROM accounts ORDER BY name")
    fun observeAccounts(): Flow<List<AccountEntity>>

    @Query("SELECT * FROM accounts WHERE id = :id")
    suspend fun getById(id: String): AccountEntity?

    @Upsert
    suspend fun upsertAll(accounts: List<AccountEntity>)

    @Query("DELETE FROM accounts")
    suspend fun clear()
}
```



©Logical Imagination Group LLC

The Database Class and In-Memory Tests

@Database lists entities, version, and exportSchema = true

Build one instance per process with Room.databaseBuilder

Tests use Room.inMemoryDatabaseBuilder: fast and isolated

Never call allowMainThreadQueries in production



©Logical Imagination Group LLC

The Database Class

```
@Database(entities = [AccountEntity::class], version = 1, exportSchema = true)
@TypeConverters(Converters::class)
abstract class PncDatabase : RoomDatabase() {
    abstract fun accountDao(): AccountDao
}
```



©Logical Imagination Group LLC

Lesson 2.1

Skill Check ✓

1. What is the main advantage of Room over raw SQLite?

A) It is a different database engine
B) Compile-time SQL verification and less boilerplate
C) It works without a device
D) It is encrypted by default

2. What does a DAO method returning Flow do?

A) Runs once
B) Re-emits whenever the queried table changes
C) Blocks the main thread
D) Syncs to the server

3. Why store money as Long cents in an entity?

A) SQLite cannot store text
B) Avoids floating point error and migrates easily
C) It is required by Room
D) It makes queries slower

4. Why use an in-memory database in tests?

A) It persists across runs
B) It is fast and isolated
C) It is encrypted
D) It needs the network



©Logical Imagination Group LLC

Relations, Transactions, and Migrations

- Model one-to-many relationships with foreign keys and @Relation
- Write and test a manual migration
- Group multi-step writes in a transaction
- Explain why destructive migration is unacceptable for banking data



©Logical Imagination Group LLC

Foreign Keys and One-to-Many Relations

Declare a ForeignKey so the database enforces integrity

CASCADE deletes children when a parent is removed

A POJO with @Embedded and @Relation loads parent and children together

Room does not auto-join; it runs a second query for relations



©Logical Imagination Group LLC

Establishing Foreign Keys

```
@Entity(
    tableName = "transactions",
    foreignKeys = [
      ForeignKey(
         entity = AccountEntity::class,
         parentColumns = ["id"], childColumns = ["accountId"],
         onDelete = ForeignKey.CASCADE
      )
    ],
    indices = [Index("accountId")]
)
data class TransactionEntity(
  /* columns */
val accountId: String)
```



©Logical Imagination Group LLC

Using One-to-Many Relationships

```
data class AccountWithTransactions(
    @Embedded val account: AccountEntity,

    @Relation(parentColumn = "id", entityColumn = "accountId")
    val transactions: List<TransactionEntity>
)
```



©Logical Imagination Group LLC

Transactions: All or Nothing

- @Transaction on DAO methods makes reads consistent
- Partial writes corrupt a sync: use one transaction per refresh
- Keep transactions short; never call the network inside one

Code Bite

```
@Dao
interface AccountDao {
   @Transaction
   @Query("SELECT * FROM accounts WHERE id =
:id")
   fun observeWithTransactions(id: String):
      Flow<AccountWithTransactions?>

}
```



©Logical Imagination Group LLC

Transactions: All or Nothing

- db.withTransaction groups several DAO calls atomically

Code Bite

```
suspend fun replaceAll(
    db: PncDatabase,
    accounts: List<AccountEntity>
) = db.withTransaction {
    db.accountDao().upsertAll(accounts)
    db.accountDao()
        .deleteMissing(accounts.map { it.id })

    }
```



©Logical Imagination Group LLC

Migrations

Every schema change increments the version

A Migration object describes how to get from version N to N+1

Export schemas to source control so migrations can be tested

fallbackToDestructiveMigration deletes user data: forbidden here



©Logical Imagination Group LLC

Setting up Migrations

```
@Database(
    entities = [AccountEntity::class, TransactionEntity::class],
    version = 2,
    autoMigrations = [
      AutoMigration(from = 1, to = 2)
    ],
    exportSchema = true
)
abstract class PncDatabase : RoomDatabase() {
   /* daos */
}
```



©Logical Imagination Group LLC

Migrations (generated code)

```
val MIGRATION_2_3 = object : Migration(2, 3) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL("ALTER TABLE accounts ADD COLUMN nickname TEXT")
        db.execSQL("UPDATE accounts SET nickname = name")
    }
}

Room.databaseBuilder(ctx, PncDatabase::class.java, "pnc.db")
    .addMigrations(MIGRATION_2_3)
    .build()
```



©Logical Imagination Group LLC

Testing Migrations

MigrationTestHelper opens a database at the old version

Insert old-shape data, run the migration, validate the new schema

One test per migration, plus one test for the full chain

Commit exported schema JSON files to source control

Lesson 2.2

Skill Check ✓

1. What does ForeignKey.CASCADE on delete do?

A) Blocks the delete
B) Deletes child rows when the parent is deleted
C) Ignores the delete
D) Copies rows

2. Why wrap replace-all in one transaction?

A) It is faster to type
B) A crash mid-way cannot leave half-updated data
C) Room requires it
D) It avoids indexes

3. Why is fallbackToDestructiveMigration unacceptable here?

A) It is slow
B) It silently deletes user data, including queued transfers
C) It needs the network
D) It is deprecated

4. What should a migration test assert beyond schema validity?

A) Nothing
B) That existing data survived
C) App size
D) Gradle duration



©Logical Imagination Group LLC

Lesson 3

Offline-First Repository

The Offline-First Repository

- Explain the single source of truth principle and why the UI observes the database
- Map DTO to entity to domain in clearly separated functions
- Implement a repository exposing observeAccounts(): Flow and refresh()
- Show stale data with a non-blocking error banner instead of a blank screen



©Logical Imagination Group LLC

Single Source of Truth

- The database is the only thing the UI reads
- The network never feeds the UI directly; it writes to the database
- Flow from Room pushes every change to the screen
- Result: the app works offline, and online behavior is the same code path

```
Composable <- ViewModel <- UseCase <- Repository <- Room (Flow)
                                          ^
                                          |  refresh(): fetch, map, upsert
                                          |
                                    Retrofit API
```



©Logical Imagination Group LLC

Repository Contract and Implementation

observeAccounts(): Flow<List<Account>> reads the database

refresh(): ApiResult<Unit> fetches and persists

Contract lives in the domain layer; implementation in data

Bind the interface to the implementation with Hilt @Binds



©Logical Imagination Group LLC

Clean Architecture Repositories

- In clean architecture, a Kotlin repository **should return a cold Flow** rather than a suspend function when dealing with data streams that change over time
- Because Flow is a cold stream, the code inside the repository's flow builder does not execute until a consumer explicitly collects it (e.g., in the ViewModel)
- Functions returning a Flow should **not** be marked with the suspend modifier
  - The function itself merely constructs and returns the stream configuration
  - The suspension happens internally during collection



©Logical Imagination Group LLC

Repository Contract

- Data methods return a cold flow
- Suspendable refresh methods are exposed

Code Bite

```
interface AccountsRepository {

  fun observeAccounts(): Flow<List<Account>>
  suspend fun refresh(): ApiResult<Unit>

}
```



©Logical Imagination Group LLC

Repository Contract and Implementation (code)

```
class OfflineFirstAccountsRepository @Inject constructor( . . . ) : AccountsRepository {

    override fun observeAccounts() =
        dao.observeAccounts().map { list -> list.map { it.toDomain() } }

    override suspend fun refresh(): ApiResult<Unit> = safeApiCall {
        val remote = api.getAccounts()
        val now = clock.millis()
        db.withTransaction {
            dao.upsertAll(remote.map { it.toEntity(now) })
            dao.deleteMissing(remote.map { it.id })
        }
    }
}
```



©Logical Imagination Group LLC

Three Models, Two Mappers

AccountDto.toEntity() sits in the data layer

AccountEntity.toDomain() sits in the data layer

Domain models never import Retrofit, Room, or serialization

Mapper tests are cheap and catch real bugs



©Logical Imagination Group LLC

Three Models, Two Mappers (code)

```
fun AccountDto.toEntity(nowMs: Long) = AccountEntity(
    id = id, name = name, maskedNumber = maskedNumber,
    balanceCents = balanceCents, updatedAtEpochMs = nowMs
)

fun AccountEntity.toDomain() = Account(
    id = id, name = name, maskedNumber = maskedNumber,
    balance = Money(balanceCents),
    lastUpdated = Instant.ofEpochMilli(updatedAtEpochMs)
)
```



©Logical Imagination Group LLC

UI State: Cached Content Plus Refresh Status

Combine the data Flow with a refresh status Flow

Show stale content immediately, then update when the network responds

Show a banner for refresh errors, never replace content with an error screen

Show last updated time so users trust what they see



©Logical Imagination Group LLC

UI State

- UI state needs to track the data relevant to the view
- It also should track the last updated date/time
  - User knows how stale data is
  - Can be factored into refresh strategies
- Error messages should be surfaced to the UI
- It also should track whether a refresh is currently under way
  - This can help avoid triggering another refresh in the middle of one
  - Or possibly cancel the existing refresh in favor of a new one

Code Bite

```
data class AccountsUi(
    val accounts: List<Account> =
                        emptyList(),
    val refreshing: Boolean = false,
    val error: String? = null,
    val lastUpdated: Instant? = null
)
```



©Logical Imagination Group LLC

UI State: Cached Content Plus Refresh Status (code)

```
@HiltViewModel
class AccountsViewModel @Inject constructor(private val repo: AccountsRepository) :
ViewModel() {
    private val status = MutableStateFlow(Pair(false, null as String?))

    val ui: StateFlow<AccountsUi> = combine(repo.observeAccounts(), status) { list, (busy,
err) ->
        AccountsUi(list, busy, err, list.maxOfOrNull { it.lastUpdated })
    }.stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), AccountsUi())

    fun refresh() = viewModelScope.launch {
        status.value = true to null
        val r = repo.refresh()
        status.value = false to (if (r is ApiResult.Success) null else r.toUserMessage())
    }
}
```



©Logical Imagination Group LLC

Lesson 3.1

Skill Check

1. In an offline-first design, what does the UI observe?

A) The network response
B) The local database
C) A global variable
D) The manifest

2. Why split observe and refresh in the repository?

A) Style preference
B) They have different lifecycles: continuous observation versus a one-time event
C) Retrofit requires it
D) To add more code

3. Where should a refresh error appear if cached data exists?

A) Full-screen error
B) A banner above the cached content
C) Nowhere
D) A crash

4. Why do mappers keep DTO, entity, and domain separate?

A) Compilation speed
B) Each layer can change independently
C) Required by Hilt
D) To confuse readers



©Logical Imagination Group LLC

Module 14 Wrap-Up and What Comes Next

- The network layer converts every failure into a value; cancellation is never swallowed.
- DTO, entity, and domain are three shapes with mappers at each boundary.
- The database is the source of truth and the UI observes it.
- Writes made offline go through an outbox with idempotency keys.
- Next: Module 15, Android Testing and Optimization



©Logical Imagination Group LLC